

建構跨雲端開放式資料湖倉

來源：<https://codelabs.developers.google.com/next26/multicloud-lakehouse?hl=zh-tw>

步驟數：11

使用 BigQuery、AlloyDB、Managed Service for Apache Spark 和 Gemini，建構跨雲端開放式資料湖倉。

目錄

1. 簡介
2. 設定和需求
3. 設定核心基礎架構
4. 佈建作業資料庫
5. 聯合主要資料 (AWS 區域)
6. 擷取事件記錄 (Google Cloud spoke)
7. 建立整合式顧客資料
8. 使用 BigQuery 資料代理程式進行分析
9. 使用 Gemini 和 MCP 生成 AI 洞察資訊
10. 清理環境
11. 恭喜！

步驟 1 · 約 5 分鐘

1. 簡介

在本程式碼研究室中，您將建構跨雲端開放式資料湖倉，整合 AWS、Google Cloud 和 AlloyDB 的資料孤島，不必進行複雜的 ETL 作業。您將使用 Lakehouse 做為中央智慧中樞、AlloyDB 做為作業資料來源，以及 Managed Service for Apache Spark 進行高效能向量化處理。最後，您將使用 Gemini 從資料湖倉中取得寶貴的業務洞察資料。

假設您的交易資料 (`users` 、 `orders` 、 `order items`) 位於 AlloyDB 作業資料庫中， `product` 資料位於 AWS S3 值區中，而大量的點擊串流 `event logs` 則儲存在 Cloud Storage 中。您需要合併這些資料集，找出下一個行銷廣告活動的目標客層，並產生個人化電子郵件。

必要條件

- 熟悉基本的 SQL 和終端機指令。
- 已啟用計費功能的 Google Cloud 專案。

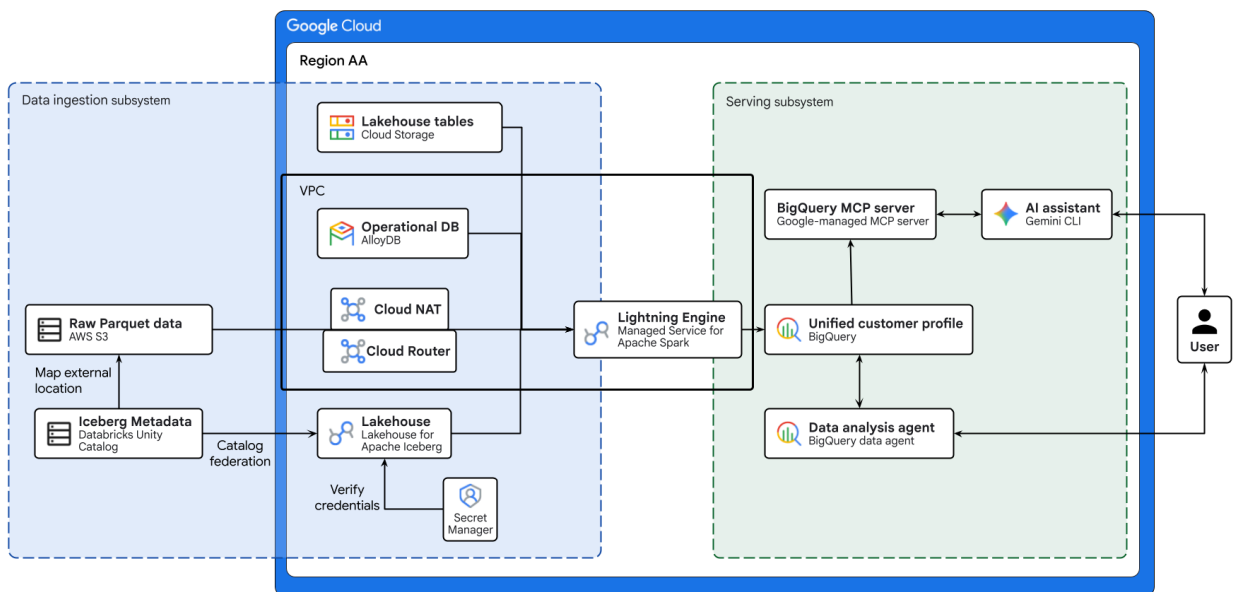
課程內容

- 如何使用 BigQuery Zero-ETL (AlloyDB) 和 Lakehouse for Apache Iceberg，整合不同的資料孤島。
- 如何使用 Managed Service for Apache Spark (由 C++ 原生 Lightning Engine 提供支援) 執行高速行為剖析工作。
- 如何使用 BigQuery 資料代理，對整合資料執行複雜的自然語言分析。
- 如何設定 Model Context Protocol (MCP)，讓 Gemini CLI 從 Apache Iceberg 的 Lakehouse 讀取資料，並草擬行銷內容。

軟硬體需求

- Google Cloud 帳戶和 Google Cloud 專案
- 網路瀏覽器，例如 Chrome

軟硬體需求



上圖說明本程式碼研究室建構的跨雲端 Lakehouse 端對端流程。AlloyDB 的作業資料和 AWS S3 的遠端資料，會透過 Lakehouse for Apache Iceberg 安全地聯合。Managed Service for Apache Spark 會處理這些不同的來源，在 BigQuery

中建立統一的客戶特徵。最後，透過 Gemini CLI 使用 BigQuery 資料代理程式和 Google 管理的 BigQuery MCP 伺服器，即可透過直覺的 AI 驅動介面進行進階資料分析。

重要概念

- **跨雲端開放式資料湖倉**：整合 AWS、Google Cloud 和地端部署環境的資料孤島，無須複雜的 ETL。
- **BigQuery 零 ETL**：可直接查詢作業資料庫，不必進行複雜的資料移動。
- **Lakehouse for Apache Iceberg**：使用 Apache Iceberg 格式，在跨雲端儲存空間中實現一致的安全性與治理。
- **Lightning Engine**：C++ 原生引擎，可執行高效能的 Apache Spark。
- **Model Context Protocol (MCP)**：將 Gemini 直接連結至 BigQuery Lakehouse。

💡 費用估算：假設您在教學課程結束時，使用提供的清除指令碼立即刪除所有資源，執行本程式碼研究室的費用將低於 \$5 美元。

步驟 2 · 約 5 分鐘

2. 設定和需求

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上角工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Cloud Shell terminal. The browser tab is titled 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal has a dark background with white text. The first line says 'Welcome to Cloud Shell! Type "help" to get started.' The second line shows the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' followed by a red cursor.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

初始化環境

開啟 Cloud Shell 並設定專案變數，確保所有指令都以正確的基礎架構為目標。

```

cat << 'EOF' > env.sh
#!/bin/bash
# env.sh: Environment variables

export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-west1"
export NETWORK_NAME="default"

export BUCKET_NAME="lakehouse-data-${PROJECT_ID}"
export BQ_DATASET="demo_lakehouse"
export BQ_RESOURCE_CONN="lakehouse-iceberg-conn"
export BQ_ALLOYDB_CONN="alloydb-fed-conn"

export ALLOYDB_CLUSTER="demo-alloy-cluster"
export ALLOYDB_INSTANCE="demo-alloy-primary"
export ALLOYDB_PASSWORD="SuperSecretPassword123!"
export ALLOYDB_DB_NAME="retail_db"

# Multi-cloud configuration identifiers
export SECRET_NAME="dbx-oauth-secret"
export CATALOG_NAME="aws_dbx_catalog"
EOF

```

將變數套用至有效工作階段：

```
source ./env.sh
```

啟用 API

啟用必要的 Google Cloud 服務。

```

gcloud services enable \
  geminidataanalytics.googleapis.com \
  cloudaicompanion.googleapis.com \
  compute.googleapis.com \
  biglake.googleapis.com \
  bigquery.googleapis.com \
  bigqueryconnection.googleapis.com \
  alloydb.googleapis.com \
  servicenetworking.googleapis.com \
  secretmanager.googleapis.com \
  dataplex.googleapis.com \
  datacatalog.googleapis.com \
  dataform.googleapis.com \
  dataproc.googleapis.com --quiet

```

3. 設定核心基礎架構

您將建構聯合跨雲端架構，而不是透過脆弱的 ETL 管道將所有資料移至單一存放區。在現實世界的企業中，由於系統需求不同，資料本質上就是分散的。您將協調下列資料來源：

- **AlloyDB (核心交易資料庫)**：儲存使用者、訂單和 `order_items` 資料。做為即時營運資料庫，它可確保金融交易和設定檔更新所需的 ACID 屬性。
- **AWS S3 (主要資料)**：儲存 `products` 目錄。在 AWS 上代表舊版主要資料管理 (MDM) 系統。
- **Google Cloud Storage (巨量資料湖泊)**：儲存 `events` (點擊串流記錄)。如果使用關聯式資料庫，高處理量資料 (例如網頁記錄) 會導致系統當機。物件儲存空間提供無限擴充性，且儲存在 Google Cloud 中可讓分析引擎的運算位置達到最佳狀態。

首先，請設定基礎網路。AlloyDB 等 Google Cloud 受管理資料庫需要私有虛擬私有雲對等互連連線，才能在專案網路中安全地通訊。

```
# Allocate an IP range for Google Cloud managed services
gcloud compute addresses create google-managed-services-${NETWORK_NAME} \
  --global \
  --purpose=VPC_PEERING \
  --prefix-length=24 \
  --network=projects/${PROJECT_ID}/global/networks/${NETWORK_NAME}

# Establish the VPC peering connection
gcloud services vpc-peerings connect \
  --service=servicenetworking.googleapis.com \
  --ranges=google-managed-services-${NETWORK_NAME} \
  --network=${NETWORK_NAME} \
  --project=${PROJECT_ID}
```

⚠ 警告：建立連線需要幾分鐘。

接著，請建立 BigQuery 資料集和 Lakehouse Cloud 資源連結。在架構上，資源連線會將資料存取權委派給專用的 Google 代管服務帳戶，並強制執行最小權限原則。

```
# Create the central data lakehouse dataset
bq mk --dataset --location=${REGION} ${PROJECT_ID}:${BQ_DATASET}
gcloud storage buckets create gs://${BUCKET_NAME} --location=${REGION}

# Create a Lakehouse resource connection
bq mk --connection --location=${REGION} \
    --connection_type=CLOUD_RESOURCE ${BQ_RESOURCE_CONN}

# Retrieve the automatically provisioned service account
CONN_SA=$(bq show --connection --format=json ${PROJECT_ID}.${REGION}.${BQ_RESOURCE_CONN} | jq
-r '.cloudResource.serviceAccountId')

# Grant the service account permissions to read/write to the data lake
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
    --member="serviceAccount:${CONN_SA}" \
    --role="roles/storage.admin" \
    --quiet
```

4. 佈建作業資料庫

佈建 AlloyDB 主要執行個體，並注入重要業務交易資料。

⚠ 警告：佈建主要 AlloyDB 執行個體約需 15 到 20 分鐘。

```
# Create the AlloyDB cluster
gcloud alloydb clusters create ${ALLOYDB_CLUSTER} \
  --region=${REGION} \
  --password=${ALLOYDB_PASSWORD} \
  --network=projects/${PROJECT_ID}/global/networks/${NETWORK_NAME}

# Create the primary instance
gcloud alloydb instances create ${ALLOYDB_INSTANCE} \
  --cluster=${ALLOYDB_CLUSTER} \
  --region=${REGION} \
  --instance-type=PRIMARY \
  --cpu-count=2 \
  --assign-inbound-public-ip=ASSIGN_IPV4 \
  --database-flags=password.enforce_complexity=on
```

資料庫準備就緒後，您必須建立 BigQuery 外部連線至 AlloyDB。這項連線會安全地儲存資料庫憑證和端點，讓 BigQuery 直接將 SQL 執行作業下推至 AlloyDB 計算引擎 (零 ETL)。

```
# Create the BigQuery to AlloyDB connection
bq mk --connection --location=${REGION} --project_id=${PROJECT_ID} \
  --connector_configuration "{
    \"connector_id\": \"google-alloydb\",
    \"asset\": {
      \"database\": \"${ALLOYDB_DB_NAME}\",
      \"google_cloud_resource\":
\"//alloydb.googleapis.com/projects/${PROJECT_ID}/locations/${REGION}/clusters/${ALLOYDB_CLUSTER}/instances/${ALLOYDB_INSTANCE}\"
    },
    \"authentication\": {
      \"username_password\": {
        \"username\": \"postgres\",
        \"password\": { \"plaintext\": \"${ALLOYDB_PASSWORD}\" }
      }
    }
  }" ${BQ_ALLOYDB_CONN}

# Grant the BigQuery connection service agent permission to access AlloyDB
PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --format="value(projectNumber)")
BQ_SERVICE_AGENT="service-${PROJECT_NUMBER}@gcp-sa-bigqueryconnection.iam.gserviceaccount.com"

gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${BQ_SERVICE_AGENT}" \
  --role="roles/alloydb.client" \
  --quiet
```

安全地將交易資料表推送至 AlloyDB。使用 AlloyDB Auth Proxy，將本機 Cloud Shell 工作階段安全地連線至私有 AlloyDB 執行個體。這樣一來，您就能使用本機指令列工具推送交易資料。

```

# Extract full raw data to Cloud Storage using the BigQuery extract API
bq extract --destination_format=CSV --print_header=true "bigquery-public-
data:thelook_ecommerce.users" gs://${BUCKET_NAME}/tmp/users.csv
bq extract --destination_format=CSV --print_header=true "bigquery-public-
data:thelook_ecommerce.orders" gs://${BUCKET_NAME}/tmp/orders.csv
bq extract --destination_format=CSV --print_header=true "bigquery-public-
data:thelook_ecommerce.order_items" gs://${BUCKET_NAME}/tmp/order_items.csv

# Download the CSVs to the local Cloud Shell session
gcloud storage cp gs://${BUCKET_NAME}/tmp/users.csv .
gcloud storage cp gs://${BUCKET_NAME}/tmp/orders.csv .
gcloud storage cp gs://${BUCKET_NAME}/tmp/order_items.csv .

# Download and start the AlloyDB auth proxy
curl -sL "https://storage.googleapis.com/alloydb-auth-proxy/v1.13.11/alloydb-auth-
proxy.linux.amd64" -o alloydb-auth-proxy && chmod +x alloydb-auth-proxy

./alloydb-auth-proxy
projects/${PROJECT_ID}/locations/${REGION}/clusters/${ALLOYDB_CLUSTER}/instances/${ALLOYDB_IN
STANCE} --public-ip &
PROXY_PID=$!
sleep 15 # Wait for the proxy to fully initialize

# Create the database
export PGPASSWORD=${ALLOYDB_PASSWORD}
psql -h 127.0.0.1 -p 5432 -U postgres -c "CREATE DATABASE ${ALLOYDB_DB_NAME};" || true

# Load into AlloyDB mimicking the exact schema via heredoc
psql -h 127.0.0.1 -p 5432 -U postgres -d ${ALLOYDB_DB_NAME} << 'EOF'
CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KEY, first_name VARCHAR(255), last_name
VARCHAR(255), email VARCHAR(255), age INT, gender VARCHAR(50), state VARCHAR(100),
street_address VARCHAR(255), postal_code VARCHAR(50), city VARCHAR(100), country
VARCHAR(100), latitude FLOAT, longitude FLOAT, traffic_source VARCHAR(100), created_at
TIMESTAMP, user_geom TEXT);

CREATE TABLE IF NOT EXISTS orders (order_id INT PRIMARY KEY, user_id INT, status VARCHAR(50),
gender VARCHAR(50), created_at TIMESTAMP, returned_at TIMESTAMP, shipped_at TIMESTAMP,
delivered_at TIMESTAMP, num_of_item INT);

CREATE TABLE IF NOT EXISTS order_items (id INT PRIMARY KEY, order_id INT, user_id INT,
product_id INT, inventory_item_id INT, status VARCHAR(50), created_at TIMESTAMP, shipped_at
TIMESTAMP, delivered_at TIMESTAMP, returned_at TIMESTAMP, sale_price FLOAT);

\copy users FROM 'users.csv' WITH (FORMAT csv, HEADER true)
\copy orders FROM 'orders.csv' WITH (FORMAT csv, HEADER true)
\copy order_items FROM 'order_items.csv' WITH (FORMAT csv, HEADER true)
EOF

# Clean up local temporary files and Cloud Storage artifacts
kill $PROXY_PID && rm -f users.csv orders.csv order_items.csv alloydb-auth-proxy
gcloud storage rm gs://${BUCKET_NAME}/tmp/*.csv

```


5. 聯合主要資料 (AWS 區域)

我們的產品目錄包含原始項目中繼資料，以 Apache Iceberg 資料表的形式原生存放在 AWS S3 中。中繼資料由遠端目錄管理。

您不必建構容易出錯的 ETL 管道，將資料複製到 Google Cloud，而是使用 **Lakehouse for Apache Iceberg (REST 目錄同盟)**。

透過這種零 ETL 方法，Lakehouse 和 Managed Service for Apache Spark 就能直接從遠端環境動態探索及讀取 Iceberg 中繼資料和基礎 Parquet 檔案。

如果您有有效的 AWS 帳戶，且已設定 Databricks Unity Catalog，即可使用該帳戶。否則，您可以使用 Google Cloud Storage 模擬環境。選擇其中一項。

選項 A：自備 AWS (原生 Apache Iceberg)

必要條件：選取這個選項前，請先佈建 AWS S3 bucket、將其連線為 Databricks Unity Catalog 中的外部位置、對應 Iceberg 資料表，並產生具備讀取權的 OAuth 服務主體。

1. 安全憑證儲存空間

將長期存取權杖硬式編碼是架構反模式。您會在 Google Cloud Secret Manager 中儲存 Databricks OAuth 用戶端 ID 和密鑰。Lakehouse 服務會在執行階段動態擷取這些憑證，以提供短期權杖，集中管理憑證。

執行下列程式碼區塊來產生指令碼。(請勿編輯任何內容)。

```
cat << 'EOF' > create_secret.sh
#!/bin/bash
source ./env.sh

# Define your Databricks OAuth credentials
DATABRICKS_CLIENT_ID=<YOUR_DATABRICKS_CLIENT_ID>
DATABRICKS_CLIENT_SECRET=<YOUR_DATABRICKS_CLIENT_SECRET>
DATABRICKS_WORKSPACE=<YOUR_WORKSPACE>.cloud.databricks.com # Exclude https://
DATABRICKS_CATALOG="google_lakehouse_catalog"

# Define the secure credentials payload
export
CLOUDSDK_API_ENDPOINT_OVERRIDES_SECRETMANAGER="https://secretmanager.${REGION}.rep.googleapis.com/"
SECRET_PAYLOAD="{ \"client_id\": \"${DATABRICKS_CLIENT_ID}\", \"client_secret\": \"${DATABRICKS_CLIENT_SECRET}\" }"

# Pipe the JSON payload into Google Cloud Secret Manager
echo "$SECRET_PAYLOAD" | gcloud secrets create ${SECRET_NAME} \
  --location=${REGION} \
  --project=${PROJECT_ID} \
  --data-file=-
EOF
```

然後執行下列指令，在終端機上方的視覺化程式碼編輯器中自動開啟產生的指令碼。

```
cloudshell edit create_secret.sh
```

1. 在編輯器中，將 `<YOUR_...>` 預留位置替換為實際的 Databricks 憑證。
2. 請確認工作區網址不包含 `https://` 或結尾斜線 (例如 `123456789.cloud.databricks.com`)。
3. 按 `Ctrl+S` 鍵 (或 Mac 上的 `Cmd+S` 鍵) 儲存檔案。
4. 返回終端機工作階段並執行指令碼：

```
source create_secret.sh
```

2. 建立聯合目錄

為簡化本程式碼研究室，您將設定目錄，以安全地遍歷公開網際網路。不過，對於生產環境工作負載，透過公用網際網路查詢大量資料集會產生不必要的輸出費用，且延遲時間難以預測。最佳做法是在 AWS 和 Google Cloud 之間設定私有 Cross-Cloud Interconnect (CCI)，大幅降低傳出費用並確保網路效能。

使用 gcloud CLI 佈建聯合 Lakehouse 目錄：

```
# Execute the Lakehouse CLI to provision the federated catalog
gcloud alpha biglake iceberg catalogs create ${CATALOG_NAME} \
  --project="${PROJECT_ID}" \
  --primary-location="${REGION}" \
  --catalog-type="federated" \
  --federated-catalog-type="unity" \
  --secret-name="projects/${PROJECT_ID}/locations/${REGION}/secrets/${SECRET_NAME}" \
  --unity-instance-name="${DATABRICKS_WORKSPACE}" \
  --unity-catalog-name="${DATABRICKS_CATALOG}" \
  --refresh-interval="330s"
```

3. 套用最低權限 IAM 繫結

在上一個步驟中佈建聯合目錄時，Google Cloud Lakehouse 會自動啟動背景重新整理作業，每 330 秒同步處理 Iceberg 資訊清單。

您必須將 `secretAccessor` 角色授予 Lakehouse 目錄服務帳戶，這樣該帳戶才能在這些背景同步和查詢執行期間，安全地擷取 Databricks OAuth 權杖。如果缺少這項繫結，Lakehouse 嘗試更新目錄時，就會發生無聲的 403 錯誤。

```
# Extract the automatically provisioned Lakehouse catalog service account
LAKEHOUSE_SA=$(curl -s -X GET
"https://biglake.googleapis.com/iceberg/v1/restcatalog/extensions/projects/${PROJECT_ID}/catalogs/${CATALOG_NAME}" \
  -H "Authorization: Bearer $(gcloud auth application-default print-access-token)" | jq -r
'."biglake-service-account"')

# Grant the secretAccessor role for background metadata synchronization and query execution
gcloud secrets add-iam-policy-binding ${SECRET_NAME} \
  --project=${PROJECT_ID} --location=${REGION} \
  --member="serviceAccount:${LAKEHOUSE_SA}" \
  --role="roles/secretmanager.secretAccessor" --quiet
```

⚠ **附註：**由於初始中繼資料擷取作業完全仰賴背景同步處理工作，因此在 Google Cloud 環境中，聯合 AWS 資料表最多需要 330 秒才能完全顯示並可供查詢。您可以繼續進行後續步驟，但在執行最終查詢時，請務必留意這點。

4. 啟用 Managed Service for Apache Spark 的輸出網際網路

在後續步驟中，Managed Service for Apache Spark 會讀取遠端 AWS 資料。由於 Managed Service for Apache Spark Serverless 完全在沒有外部 IP 位址的私人虛擬私有雲網路中執行，因此預設無法透過網際網路連線至 AWS S3。您必須佈建 Cloud NAT，才能允許 Spark 工作站連出網際網路。

```
# Create a Cloud Router
gcloud compute routers create lakehouse-router \
  --network=${NETWORK_NAME} \
  --region=${REGION}

# Create a Cloud NAT attached to the router
gcloud compute routers nats create lakehouse-nat \
  --router=lakehouse-router \
  --auto-allocate-nat-external-ips \
  --nat-all-subnet-ip-ranges \
  --region=${REGION}
```

5. 定義下游目標

匯出這個變數，讓下游 Apache Spark 工作確切知道要查詢 AWS 資料的位置，不必手動變更程式碼。

```
# Assuming your schema is 'retail' and table is 'aws_products'
export AWS_PRODUCTS_TABLE="${CATALOG_NAME}.retail.aws_products"

# Persist the variable for future shell sessions
echo "export AWS_PRODUCTS_TABLE=\"${AWS_PRODUCTS_TABLE}\"" >> env.sh
```

選項 B：透過 Cloud Storage 模擬 AWS 環境

如果您沒有有效的 AWS 帳戶，可以使用 Google Cloud Storage 上的 Lakehouse 代管資料表，在本機模擬跨雲端資料孤島。

1. 建立模擬 Iceberg 資料表

```
# Copy raw products data to a temporary BigQuery table
bq cp --force bigquery-public-data:thelook_ecommerce.products
${PROJECT_ID}:${BQ_DATASET}.temp_products_raw

# Create an open Iceberg table using the Lakehouse cloud resource connection
bq query --use_legacy_sql=false "
CREATE OR REPLACE TABLE \`${PROJECT_ID}.${BQ_DATASET}.aws_products\`
WITH CONNECTION \`${REGION}.${BQ_RESOURCE_CONN}\`
OPTIONS (file_format = 'PARQUET', table_format = 'ICEBERG', storage_uri =
'gs://${BUCKET_NAME}/aws_products')
AS SELECT * FROM \`${PROJECT_ID}.${BQ_DATASET}.temp_products_raw\`;"

# Cleanup temporary table
bq rm -f -t ${PROJECT_ID}:${BQ_DATASET}.temp_products_raw
```

2. 定義下游目標

標準 BigQuery 資料集使用 3 部分的命名空間結構 (`project.dataset.table`)。請匯出這個變數，讓下游 Apache Spark 工作以模擬資料為目標。

```
export AWS_PRODUCTS_TABLE="${PROJECT_ID}.${BQ_DATASET}.aws_products"

# Persist the variable for future shell sessions
echo "export AWS_PRODUCTS_TABLE=\"${AWS_PRODUCTS_TABLE}\"" >> env.sh
```

步驟 6 · 約 5 分鐘

6. 擷取事件記錄 (Google Cloud spoke)

點擊串流資料呈指數級成長。您會在 Cloud Storage 中以代管 Lakehouse 資料表的形式，在本機儲存完整且未經彙整的原始事件。

```
bq cp --force bigquery-public-data:thelook_ecommerce.events
${PROJECT_ID}:${BQ_DATASET}.temp_events_raw

bq query --use_legacy_sql=false "
CREATE OR REPLACE TABLE \`${PROJECT_ID}.${BQ_DATASET}.google_events\`
WITH CONNECTION \`${REGION}.${BQ_RESOURCE_CONN}\`
OPTIONS (file_format = 'PARQUET', table_format = 'ICEBERG', storage_uri =
'gs://${BUCKET_NAME}/google_events')
AS SELECT * FROM \`${PROJECT_ID}.${BQ_DATASET}.temp_events_raw\`;"

bq rm -f -t ${PROJECT_ID}:${BQ_DATASET}.temp_events_raw
```

7. 建立整合式顧客資料

原始基礎架構完全填入資料後，就可以開始建構整合式顧客設定檔。

您將使用 **Lightning Engine** 支援的 **Managed Service for Apache Spark**。Lightning Engine 是 Google Cloud 的高效能 C++ 原生查詢加速器，以 Apache Gluten 和 Velox 等開放原始碼技術為基礎，可自動提升執行效能，充分發揮 CPU 效率並智慧快取資料。如果您要執行大規模多向聯結、複雜的視窗作業，或跨多個雲端進行行為匯總，這個做法就很實用。

您將使用 **Spark BigQuery 連接器**，直接讀取聯合 AlloyDB 零 ETL 查詢和 Lakehouse 資料表，在 Spark 中以原生方式執行大量向量化匯總作業，並將產生的統一設定檔寫回 BigQuery。

設定 Managed Service for Apache Spark 的 IAM 權限

根據預設，無伺服器 Spark 會使用 Compute Engine 預設服務帳戶執行批次工作。提交工作前，您必須授予這個服務帳戶執行工作負載和管理 BigQuery 工作所需的權限。

(注意：服務名稱已變更為 *Managed Service for Apache Spark*，以反映業界標準術語，但基礎 API 指令和 IAM 角色仍使用 *dataproc* 識別碼)。

```
# Retrieve the project number to construct the Compute Engine default service account
PROJECT_NUMBER=$(gcloud projects describe ${PROJECT_ID} --format="value(projectNumber)")
export COMPUTE_SA="${PROJECT_NUMBER}-compute@developer.gserviceaccount.com"

# Grant the Managed Service for Apache Spark Worker role to allow job execution
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/dataproc.worker" \
  --quiet

# Grant the BigQuery Admin role to allow reading, writing, and querying external connections
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/bigquery.admin" \
  --quiet
```

建立及提交工作

首先，請建立 PySpark 工作指令碼。這個指令碼會根據您的 `AWS_PRODUCTS_TABLE` 環境變數，自動偵測您選擇的是**選項 A (AWS Federated Catalog)** 還是**選項 B (Google Cloud Mock)**，定義 Spark SQL 邏輯，並運用 Spark 的原生陣列操控功能計算 RFM (回訪率、頻率、獲利金額) 視窗。

在 Cloud Shell 中執行下列程式碼區塊。

```

# Determine which option was selected based on the AWS_PRODUCTS_TABLE variable
if [[ "${AWS_PRODUCTS_TABLE}" == "${CATALOG_NAME:-undefined}"* ]]; then
    echo "=> Option A (AWS Federated Catalog) detected."
    export IS_AWS_CATALOG="True"
    export OAUTH_TOKEN=$(gcloud auth print-access-token)
else
    echo "=> Option B (Google Cloud Mock) detected."
    export IS_AWS_CATALOG="False"
    export OAUTH_TOKEN=""
fi

# Create the PySpark script with safely injected variables
cat << EOF > spark_lakehouse_join.py
from pyspark.sql import SparkSession

# --- Environment Variables dynamically injected ---
PROJECT_ID = "${PROJECT_ID}"
CATALOG_NAME = "${CATALOG_NAME}"
OAUTH_TOKEN = "${OAUTH_TOKEN}"
BUCKET_NAME = "${BUCKET_NAME}"
BQ_DATASET = "${BQ_DATASET}"
REGION = "${REGION}"
BQ_ALLOYDB_CONN = "${BQ_ALLOYDB_CONN}"
AWS_PRODUCTS_TABLE = "${AWS_PRODUCTS_TABLE}"
IS_AWS_CATALOG = ${IS_AWS_CATALOG}
# -----

# 1. Initialize SparkSession (Dynamic Configuration)
packages =[
    "org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.4.3",
    "org.apache.iceberg:iceberg-aws-bundle:1.4.3",
    "org.apache.hadoop:hadoop-aws:3.3.4",
    "com.amazonaws:aws-java-sdk-bundle:1.12.262",
    "com.google.cloud.spark:spark-bigquery-with-dependencies_2.12:0.36.1"
]

builder = SparkSession.builder \
    .config("spark.sql.extensions",
"org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions") \
    .config("spark.dataproc.lightningEngine.runtime", "native") \
    .config("spark.hadoop.fs.gs.velox.client.table-cache-max-size", "0") \
    .config("spark.jars.packages", ",".join(packages))

# Conditionally configure Lakehouse REST Catalog for Option A
if IS_AWS_CATALOG:
    builder = builder \
        .config(f"spark.sql.catalog.{CATALOG_NAME}", "org.apache.iceberg.spark.SparkCatalog")
\
    .config(f"spark.sql.catalog.{CATALOG_NAME}.type", "rest") \
    .config(f"spark.sql.catalog.{CATALOG_NAME}.uri",
"https://biglake.googleapis.com/iceberg/v1/restcatalog") \
    .config(f"spark.sql.catalog.{CATALOG_NAME}.warehouse",

```

```

f"bl://projects/{PROJECT_ID}/catalogs/{CATALOG_NAME}") \\
    .config(f"spark.sql.catalog.{CATALOG_NAME}.io-impl",
"org.apache.iceberg.aws.s3.S3FileIO") \\
    .config(f"spark.sql.catalog.{CATALOG_NAME}.header.X-Iceberg-Access-Delegation",
"vended-credentials") \\
    .config(f"spark.sql.catalog.{CATALOG_NAME}.header.x-goog-user-project", PROJECT_ID)
\\
    .config(f"spark.sql.catalog.{CATALOG_NAME}.header.Authorization", f"Bearer
{OAUTH_TOKEN}") \\
    .config(f"spark.sql.catalog.{CATALOG_NAME}.rest-metrics-reporting-enabled", "false")

spark = builder.getOrCreate()

spark.conf.set("temporaryGcsBucket", BUCKET_NAME)
spark.conf.set("viewsEnabled", "true")
spark.conf.set("materializationDataset", BQ_DATASET)

# 2. Extract operational data via AlloyDB Zero-ETL
users_df = spark.read.format("bigquery").option("query", f""SELECT id AS user_id, age,
country FROM EXTERNAL_QUERY("{REGION}.{BQ_ALLOYDB_CONN}", "SELECT id, age, country FROM
users")""").load()
orders_df = spark.read.format("bigquery").option("query", f""SELECT user_id, order_id,
CAST(created_at AS TIMESTAMP) AS order_date FROM EXTERNAL_QUERY("{REGION}.{BQ_ALLOYDB_CONN}",
"SELECT user_id, order_id, created_at FROM orders WHERE status = 'Complete'')""").load()
items_df = spark.read.format("bigquery").option("query", f""SELECT order_id, product_id,
sale_price FROM EXTERNAL_QUERY("{REGION}.{BQ_ALLOYDB_CONN}", "SELECT order_id, product_id,
sale_price FROM order_items")""").load()

# 3. Read AWS Products (Option A vs B) & Google Cloud Logs
if IS_AWS_CATALOG:
    products_df = spark.table(AWS_PRODUCTS_TABLE)
else:
    products_df = spark.read.format("bigquery").option("table", AWS_PRODUCTS_TABLE).load()

events_df = spark.read.format("bigquery").option("table", f"{PROJECT_ID}.
{BQ_DATASET}.google_events").load()

# Register Temp Views
users_df.createOrReplaceTempView("live_user_profiles")
orders_df.createOrReplaceTempView("live_transactions")
items_df.createOrReplaceTempView("live_order_items")
products_df.createOrReplaceTempView("raw_aws_products")
events_df.createOrReplaceTempView("google_events")

# 4. Multi-cloud Distributed Join
unified_profile_df = spark.sql("""
WITH aws_master_catalog AS (
    SELECT id AS product_id, category, brand
    FROM raw_aws_products
),
google_behavioral_logs AS (
    SELECT user_id,

```

```

        COUNT(DISTINCT session_id) AS total_sessions,
        COUNT(CASE WHEN event_type = 'cart' THEN 1 END) AS cart_adds
FROM google_events
WHERE user_id IS NOT NULL
GROUP BY user_id
),
user_purchases AS (
    SELECT
        t.user_id, t.order_date, t.order_id, oi.sale_price,
        COALESCE(p.category, 'Unknown') AS category,
        COALESCE(p.brand, 'Unknown') AS brand
    FROM live_transactions t
    JOIN live_order_items oi ON t.order_id = oi.order_id
    LEFT JOIN aws_master_catalog p ON oi.product_id = p.product_id
),
rfm_base AS (
    SELECT
        user_id,
        MAX(order_date) AS last_purchase_date,
        COUNT(DISTINCT order_id) AS total_orders,
        SUM(sale_price) AS lifetime_value
    FROM user_purchases
    GROUP BY user_id
),
ranked_items AS (
    SELECT
        user_id, category, brand, sale_price,
        ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY sale_price DESC) as rn
    FROM user_purchases
),
top_items AS (
    SELECT
        user_id,
        COLLECT_LIST(NAMED_STRUCT('category', category, 'brand', brand, 'sale_price',
sale_price)) AS top_preferences
    FROM ranked_items
    WHERE rn <= 3
    GROUP BY user_id
)
SELECT
    CURRENT_TIMESTAMP() AS snapshot_date,
    u.user_id, u.age, u.country,
    r.last_purchase_date,
    COALESCE(r.total_orders, 0) AS total_orders,
    COALESCE(ROUND(r.lifetime_value, 2), 0.0) AS lifetime_value,
    COALESCE(b.total_sessions, 0) AS total_sessions,
    COALESCE(b.cart_adds, 0) AS cart_adds,
    t.top_preferences
FROM live_user_profiles u
LEFT JOIN rfm_base r ON u.user_id = r.user_id
LEFT JOIN top_items t ON u.user_id = t.user_id
LEFT JOIN google_behavioral_logs b ON u.user_id = b.user_id

```

```

""" )

unified_profile_df.show()

# 5. Write back to BigQuery native partitioned table
(unified_profile_df.write
 .format("bigquery")
 .option("table", f"{PROJECT_ID}.{BQ_DATASET}.unified_customer_profile")
 .option("partitionField", "snapshot_date")
 .option("partitionType", "DAY")
 .option("writeMethod", "direct")
 .mode("overwrite")
 .save())

EOF

```

指令碼完全組裝完成，且動態插入必要設定後，請將批次工作提交至 Managed Apache Spark Serverless。

```

gcloud dataproc batches submit pyspark spark_lakehouse_join.py \
  --project=${PROJECT_ID} \
  --region=${REGION} \
  --version=2.3 \
  --subnet=${NETWORK_NAME} \
  --deps-bucket=${BUCKET_NAME} \
  --properties="dataproc.tier=premium,spark.dataproc.lightningEngine.runtime=native"

```

在控制台中驗證工作執行情況

提交批次工作後，您可以確認工作是否使用加速 C++ 執行引擎：

1. 在 Google Cloud 控制台中，依序前往「Managed Service for Apache Spark」 > 「Serverless」 > 「Batches」。
2. 按一下目前正在執行的工作。
3. 在工作詳細資料窗格中，確認「層級」屬性設為 **Premium**，「引擎」設為 **Lightning Engine**。

8. 使用 BigQuery 資料代理程式進行分析

您已使用 Managed Service for Apache Spark 整合分散在不同雲端的資料，並執行大量行為匯總作業，接下來的步驟是進行資料分析。

首先，請在 BigQuery UI 中檢查新建立的統合設定檔資料表結構定義，以視覺化方式瞭解您向代理程式公開的資料結構：

1. 前往 Google Cloud 控制台的「BigQuery」[BigQuery](#)頁面。
2. 在左側的「Explorer」窗格中，展開專案和 `demo_lakehouse` 資料集。
3. 點選「`unified_customer_profile`」資料表。
4. 在主要工作區中選取「結構定義」分頁標籤。

檢查新資料表的結構定義。`top_preferences` 欄是 `REPEATED STRUCT` (包含 `category`、`brand` 和 `sale_price` 的記錄陣列)。傳統上，查詢巢狀陣列需要使用 `UNNEST()` 函式的複雜 SQL，這對業務分析師來說可能是一大障礙。將 BigQuery 資料代理程式以這個特定資料表為基礎，代理程式就會瞭解結構定義，並在幕後處理複雜的 Google 標準 SQL 作業。

建立資料代理

在本節中，您將建立 BigQuery 資料代理程式並與之互動。您不必手動編寫複雜的 SQL 來取消巢狀陣列並計算指標，而是會佈建專為新建立的統合設定檔設計的 AI 代理程式，以自然語言探索資料。

1. 在左側導覽窗格中，找到並按一下「代理商」。
2. 按一下「+ 新增代理程式」，初始化新的 AI 助理。
3. 設定代理程式：

- **代理程式名稱：** `Retail VIP Analysis Agent`
- **資料來源：** 按一下「新增來源」，然後搜尋 `unified_customer_profile` 資料表。

4. 按一下「新增」，等待幾秒鐘，讓 Agent 初始化工作區。

建立代理程式後，定義明確的**系統指令**是重要的資料治理做法。將系統指令當做語意層。透過嵌入企業範圍的業務定義、處理結構定義複雜性，以及建立分析防護措施，您可以從終端使用者身上抽象化技術複雜性，並防止 LLM 從統計上不顯著的資料得出結論。

將下列內容貼到「指令」欄位：

You are an expert Data Analyst specializing in e-commerce customer retention. Your primary data source is the `unified\_customer\_profile` table.

Strict Schema Rules:

- The `top\_preferences` column is a REPEATED STRUCT (ARRAY).
- Whenever you analyze product categories, brands, or prices, you MUST explicitly use the UNNEST() function on `top\_preferences` to access the underlying fields.

Semantic Layer & Business Definitions:

- "At-Risk VIP": Define this specific user cohort as anyone meeting ALL of the following criteria: `lifetime\_value` > 100, `cart\_adds` > 0, and `last\_purchase\_date` is more than 90 days ago.

Analytical Guardrails:

- Prioritize statistical significance. When generating business insights based on geographic or demographic groupings, explicitly ignore or deprioritize segments with negligible sample sizes (e.g., countries with very few users) to prevent skewed marketing strategies.

提示代理

由於複雜的商業邏輯 (「高風險 VIP」的確切定義) 和結構定義處理需求受到系統指令安全控管，資料分析師不需要撰寫冗長且條件繁多的提示。

在對話介面中輸入下列提示詞：

Find the total count of At-Risk VIPs grouped by country. For each country, extract the single most frequent product category based on their top preferences. Order the results by the user count in descending order.

評估生成的洞察

💡 注意：大型語言模型 (LLM) 本質上具有不確定性，且查詢會根據 `CURRENT_TIMESTAMP` 計算動態 90 天閒置期，因此每次執行查詢時，您看到的確切措辭、使用者人數和國家/地區排名會略有不同。請著重評估代理程式如何套用控管規則，而非尋找完全相符的文字。

提交提示後，請仔細檢查系統原生生成的輸出內容，評估 BigQuery Data Agent 如何強制執行系統指令，同時做為受控語意層和分析防護措施。

首先，請閱讀資料上方產生的摘要文字。請注意，代理程式會自動將您簡單的「高風險 VIP」要求，轉換為系統指令中定義的確切指標門檻 (例如參照 `lifetime_value > 100`, `cart_adds > 0`，以及 90 天的閒置時間)。這表示代理程式已將您的商業邏輯內化，因此使用者在日常提示中，完全不需要記憶或硬式編碼複雜的邏輯。

接著，展開 SQL 檢視畫面，檢查產生的程式碼。代理程式應根據您的指令，建構出數學上正確的 Google 標準 SQL：

- **動態時間範圍：**在 `WHERE` 子句中尋找時間戳記計算 (通常使用 `TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 90 DAY)`)。
- **嚴格遵守結構定義：**將 `UNNEST()` 函式明確套用至 `top_preferences` 陣列，確認代理程式遵守嚴格的結構定義規則。如要準確找出每個國家/地區最常出現的單一類別，通常會看到它使用進階技術，例如一般資料表運算式 (CTE) 中的 `ROW_NUMBER() OVER() window` 函式。

查看自動算繪的圖表和資料表。資料會以視覺化方式呈現核心留存率市場 (通常會醒目顯示中國或美國等高用量國家/地區)，以及這些市場普遍偏好的產品 (通常是「牛仔褲」)。請注意，原生 UI 會建構輸出內容，供立即使用，不需要明確的視覺化程式碼。

詳閱代理程式生成的**洞察文字** (以項目符號列出)。由於您已建立分析防護措施，請特別留意有關**資料品質或統計顯著性**的洞察資訊。如果表格底部的國家/地區使用者人數極少 (例如只有少數使用者)，代理商可能會明確標示這些國家/地區。代理程式不會根據單一資料點盲目生成目標行銷廣告活動，而是會正確指出這些異常狀況在大型策略中統計上並不顯著。這項功能可直接將治理防護措施嵌入代理程式，有效防止 AI 驅動的業務誤算。

9. 使用 Gemini 和 MCP 生成 AI 洞察資訊

代理程式已成功找出主要目標客層：特定國家/地區的風險 VIP。不過，分析師的工作只到洞察為止。如要再次吸引這些使用者，行銷團隊必須執行廣告活動。

您將使用 Model Context Protocol (MCP)，直接將外部 AI 助理連結至 BigQuery 中的特定客層名單，不必建構自訂 API，即可從資料分析轉為 AI 輔助行動。

設定 BigQuery MCP 伺服器

執行下列程式碼區塊，產生 mcp.json 設定檔。這個檔案會提供必要的連線參數，讓 Gemini CLI 能安全地與 BigQuery 介接。

```
mkdir -p ~/.gemini

cat << EOF > ~/.gemini/settings.json
{
  "mcpServers": {
    "bigquery": {
      "httpUrl": "https://bigquery.googleapis.com/mcp",
      "authProviderType": "google_credentials",
      "oauth": {
        "scopes": [
          "https://www.googleapis.com/auth/bigquery"
        ]
      }
    }
  }
}
EOF
```

生成行銷電子郵件

從終端機啟動 Gemini CLI 工具，明確傳遞 MCP 旗標，允許工具讀取 BigQuery 湖倉。

執行 Gemini CLI。

⚠ 注意：這是您第一次連線，CLI 會提示您信任本機目錄。您必須核准連結。」

```
source env.sh
gemini
```

```
> /mcp

⌚ MCP servers are starting up (1 initializing)...
Note: First startup may take longer. Tool availability will update automatically.

Configured MCP servers:

🟢 bigquery - Ready (6 tools)
Tools:
- mcp_bigquery_execute_sql
- mcp_bigquery_execute_sql_readonly
- mcp_bigquery_get_dataset_info
- mcp_bigquery_get_table_info
- mcp_bigquery_list_dataset_ids
- mcp_bigquery_list_table_ids
```

開啟提示後，請 Gemini 為目標客層草擬個人化宣傳內容：

Analyze the demo_lakehouse.unified_customer_profile table in BigQuery. Find exactly one 'country and age group' target demographic that has a high average order value (sale_price >= \$80) but a relatively low total revenue compared to others. Then, draft a highly engaging, premium VIP promotional email tailored to this specific demographic. Use \$PROJECT_ID to get the current project id.

Gemini 會透過 MCP 伺服器自動查詢 BigQuery、找出目標客層，並為您草擬電子郵件！

10. 清理環境

如要避免系統持續向您的 Google Cloud 帳戶收取費用，並為日後執行作業乾淨地重設專案，請務必刪除本程式碼研究室期間建立的資源。

執行下列程式碼區塊，建立 `cleanup.sh` 指令碼。這個指令碼會做為自動拆除機制，永久移除 AlloyDB 叢集和執行個體、BigQuery 資料集和 Cloud Storage 值區，避免產生後續費用。

⚠ 共用專案的警告：下方的清除指令碼也會刪除服務網路 VPC 對等互連連線 (`servicenetworking.googleapis.com`)。如果您使用可拋棄的獨立程式碼研究室專案，建議您這麼做，完全沒有安全疑慮。不過，如果您是在共用或實際運作的 Google Cloud 專案中執行這項作業，**刪除對等互連會立即中斷其他共用相同 VPC 的受管理資料庫 (例如 Cloud SQL 或 Memorystore) 連線**。如果其他服務依賴這個虛擬私有雲，請先從指令碼中移除 `peerings delete` 行，再執行指令碼。

```

cat << 'EOF' > cleanup.sh
#!/bin/bash
source ./env.sh

echo "> Deleting BigQuery Dataset (${BQ_DATASET})..."
bq rm -r -f -d ${PROJECT_ID}:${BQ_DATASET} || true

echo "> Deleting Lakehouse resource connection (${BQ_RESOURCE_CONN})..."
bq rm --connection --project_id=${PROJECT_ID} --location=${REGION} ${BQ_RESOURCE_CONN} ||
true

echo "> Deleting AlloyDB connection (${BQ_ALLOYDB_CONN})..."
bq rm --connection --project_id=${PROJECT_ID} --location=${REGION} ${BQ_ALLOYDB_CONN} || true

echo "> Deleting AlloyDB Instance (${ALLOYDB_INSTANCE}) - This takes a few minutes..."
gcloud alloydb instances delete ${ALLOYDB_INSTANCE} --cluster=${ALLOYDB_CLUSTER} --
region=${REGION} --quiet || true

echo "> Deleting AlloyDB Cluster (${ALLOYDB_CLUSTER})..."
gcloud alloydb clusters delete ${ALLOYDB_CLUSTER} --region=${REGION} --force --quiet || true

echo "> Deleting Cloud Storage bucket (${BUCKET_NAME})..."
gcloud storage rm -r gs://${BUCKET_NAME} || true

echo "> Deleting Lakehouse Federated Catalog (if created in Option A)..."
curl -s -X DELETE
"https://biglake.googleapis.com/iceberg/v1/restcatalog/extensions/projects/${PROJECT_ID}/cata
logs/aws_dbx_catalog" \
-H "Authorization: Bearer $(gcloud auth application-default print-access-token)" || true

echo "> Deleting Secret Manager Regional Secret (if created in Option A)..."
CLOUDSDK_API_ENDPOINT_OVERRIDES_SECRETMANAGER="https://secretmanager.${REGION}.rep.googleapis
.com/" \
gcloud secrets delete dbx-oauth-secret --location=${REGION} --project=${PROJECT_ID} --quiet
|| true

echo "> Deleting Cloud NAT and Router (if created in Option A)..."
gcloud compute routers nats delete lakehouse-nat --router=lakehouse-router --region=${REGION}
--quiet || true
gcloud compute routers delete lakehouse-router --region=${REGION} --quiet || true

echo "> Deleting Service Networking VPC Peering..."
gcloud compute networks peerings delete servicenetworking-googleapis-com \
--network=${NETWORK_NAME} \
--project=${PROJECT_ID} --quiet || true

echo "> Deleting Allocated IP Range for Managed Services..."
gcloud compute addresses delete google-managed-services-${NETWORK_NAME} \
--global \
--project=${PROJECT_ID} --quiet || true

echo "> Removing local temporary files..."

```

```
rm -f spark_lakehouse_join.py users.csv orders.csv order_items.csv alloydb-auth-proxy env.sh
cleanup.sh ~/.gemini/settings.json

echo "=====
echo "  TEARDOWN COMPLETED."
echo "=====
EOF
```

執行清除指令碼，安全地刪除資源：

```
bash cleanup.sh
```

步驟 11 · 約 5 分鐘

11. 恭喜！

您已成功建構及查詢跨雲端開放式資料湖倉。

並瞭解：

- 瞭解如何使用 BigQuery Zero-ETL 和 Google Cloud Lakehouse，從不同來源聯合資料。
- 瞭解如何在 Managed Service for Apache Spark 的大量向量化聯結中，運用 C++ 原生 Lightning Engine。
- 如何使用 BigQuery 資料代理以自然語言探索資料。
- 如何使用 Model Context Protocol (MCP) 和 Gemini，將資料橋接至 Gemini。

後續步驟

- 瀏覽 [Lakehouse 說明文件](#)
 - 進一步瞭解 [AlloyDB 無須 ETL 即可將資料匯出至 BigQuery](#)
 - 瞭解 [Lightning Engine](#)
-